```
import { useEffect, useState } from 'react';
import { useRouter } from 'next/router';
import Link from 'next/link';
import { supabase } from '../../lib/supabase';

const QUESTION_COUNT = 10;
const TIME_PER_QUESTION = 30;

function FlagImage({ iso }) {
  return (
    <div style={{
      width: '100%',
      display: 'flex',
      justifyContent: 'center',
      alignItems: 'center',
      padding: '1.5rem',
      background: 'linear-gradient(135deg, #f8fafc, #e2e8f0)',
      borderRadius: '12px',
      marginBottom: '1.5rem',
      border: '2px solid #e2e8f0',
    }}>
      <img
        src={`https://flagcdn.com/w320/${iso}.png`}
        alt="Steag"
        style={{
          maxWidth: '280px',
          width: '100%',
          height: 'auto',
          borderRadius: '6px',
          boxShadow: '0 4px 12px rgba(0,0,0,0.15)',
        }}
      />
    </div>
  );
}

function HintMap({ lat, lng, radius }) {
  useEffect(() => {
    if (typeof window === 'undefined') return;
    if (!document.getElementById('leaflet-css')) {
      const link = document.createElement('link');
      link.id = 'leaflet-css';
      link.rel = 'stylesheet';
      link.href = 'https://unpkg.com/leaflet@1.9.4/dist/leaflet.css';
```

```
      document.head.appendChild(link);
    }
    if (!window.L) {
      const script = document.createElement('script');
      script.src = 'https://unpkg.com/leaflet@1.9.4/dist/leaflet.js';
      script.onload = () => initMap();
      document.head.appendChild(script);
    } else {
      initMap();
    }
    function initMap() {
      const container = document.getElementById('hint-map');
      if (!container || !window.L) return;
      if (container._leaflet_id) {
        container._leaflet_id = null;
        container.innerHTML = '';
      }
      const map = window.L.map('hint-map', {
        zoomControl: true,
        attributionControl: false,
        scrollWheelZoom: false,
      }).setView([lat, lng], getZoomFromRadius(radius));
      window.L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        maxZoom: 18,
        minZoom: 2,
      }).addTo(map);
      window.L.circle([lat, lng], {
        color: '#3b82f6',
        fillColor: '#3b82f6',
        fillOpacity: 0.2,
        weight: 2,
        dashArray: '8, 4',
        radius: radius,
      }).addTo(map);
    }
  }, [lat, lng, radius]);
  return (
    <div style={{ width: '100%', height: '300px', borderRadius: '12px', overflow:
      <div id="hint-map" style={{ width: '100%', height: '100%' }}></div>
      <div style={{ position: 'absolute', top: 8, left: 8, background: 'rgba(255,
        Indiciu vizual aproximativ
      </div>
    </div>
  );
}

function getZoomFromRadius(radius) {
```

```
  if (radius >= 2000000) return 2;
  if (radius >= 1000000) return 3;
  if (radius >= 500000) return 4;
  if (radius >= 200000) return 5;
  return 6;
}

export default function PlayQuiz() {
  const router = useRouter();
  const { chapterId } = router.query;
  const [chapter, setChapter] = useState(null);
  const [questions, setQuestions] = useState([]);
  const [currentIdx, setCurrentIdx] = useState(0);
  const [score, setScore] = useState(0);
  const [correct, setCorrect] = useState(0);
  const [selectedOption, setSelectedOption] = useState(null);
  const [fillAnswer, setFillAnswer] = useState('');
  const [showResult, setShowResult] = useState(false);
  const [isCorrect, setIsCorrect] = useState(false);
  const [timeLeft, setTimeLeft] = useState(TIME_PER_QUESTION);
  const [loading, setLoading] = useState(true);
  const [finished, setFinished] = useState(false);
  const [startTime, setStartTime] = useState(Date.now());

  useEffect(() => {
    if (!chapterId) return;
    loadQuiz();
  }, [chapterId]);

  async function loadQuiz() {
    const { data: chap } = await supabase.from('chapters').select('*').eq('id', c
    setChapter(chap);
    const { data: qs } = await supabase.from('questions').select('*').eq('chapter
    if (qs && qs.length > 0) {
      const shuffled = [...qs].sort(() => Math.random() - 0.5).slice(0, QUESTION_
      setQuestions(shuffled);
    }
    setLoading(false);
    setStartTime(Date.now());
  }

  useEffect(() => {
    if (loading || showResult || finished) return;
    if (timeLeft <= 0) { handleAnswer(null); return; }
    const t = setTimeout(() => setTimeLeft(timeLeft - 1), 1000);
    return () => clearTimeout(t);
  }, [timeLeft, loading, showResult, finished]);
```

```
function handleAnswer(answer) {
  if (showResult) return;
  const q = questions[currentIdx];
  let isC = false;
  if (q.type === 'multiple_choice') {
    isC = answer === q.correct_answer;
  } else {
    const userAns = (answer || '').trim().toLowerCase();
    let accepted = q.accepted_answers;
    if (typeof accepted === 'string') {
      accepted = accepted.replace(/[{}"]/g, '').split(',').map(s => s.trim());
    }
    if (!accepted) accepted = [q.correct_answer];
    isC = accepted.some(a => a.toLowerCase().trim() === userAns);
  }
  setIsCorrect(isC);
  setSelectedOption(answer);
  setShowResult(true);
  if (isC) {
    const points = (q.level === 'Avansat' ? 20 : 10) + Math.floor(timeLeft / 3)
    setScore(s => s + points);
    setCorrect(c => c + 1);
  }
}

async function nextQuestion() {
  if (currentIdx + 1 >= questions.length) {
    await saveResults();
    setFinished(true);
  } else {
    setCurrentIdx(i => i + 1);
    setSelectedOption(null);
    setFillAnswer('');
    setShowResult(false);
    setTimeLeft(TIME_PER_QUESTION);
  }
}

async function saveResults() {
  const { data: { session } } = await supabase.auth.getSession();
  if (!session) return;
  const duration = Math.floor((Date.now() - startTime) / 1000);
  await supabase.from('game_results').insert({
    user_id: session.user.id,
    chapter_id: parseInt(chapterId),
    score,
```

```
    questions_correct: correct,
    questions_total: questions.length,
    duration_seconds: duration
  });
  const { data: profile } = await supabase.from('profiles').select('total_score
  if (profile) {
    await supabase.from('profiles').update({
      total_score: (profile.total_score || 0) + score,
      games_played: (profile.games_played || 0) + 1
    }).eq('id', session.user.id);
  }
}

if (loading) return <div className="loading container">Se incarca intrebarile..
if (!questions.length) {
  return (
    <div className="container" style={{padding:'4rem 0', textAlign:'center'}}>
      <h2>Nu s-au gasit intrebari pentru acest capitol.</h2>
      <Link href="/" className="btn btn-primary" style={{marginTop:'1rem'}}>Ina
    </div>
  );
}

if (finished) {
  const pct = Math.round((correct / questions.length) * 100);
  return (
    <div className="container">
      <div className="results">
        <div style={{fontSize:'4rem'}}>{pct >= 80 ? 'Excelent' : pct >= 60 ? 'B
        <div className="score">{score}</div>
        <div className="score-label">puncte castigate</div>
        <div className="stats">
          <div className="stat"><div className="v">{correct}/{questions.length}
          <div className="stat"><div className="v">{pct}%</div><div className="
          <div className="stat"><div className="v">{chapter?.emoji}</div><div c
        </div>
        <div className="actions">
          <button onClick={() => router.reload()} className="btn btn-primary">M
          <Link href="/" className="btn btn-outline" style={{borderColor:'var(-
          <Link href="/leaderboard" className="btn btn-success">Clasament</Link
        </div>
      </div>
    </div>
  );
}

const q = questions[currentIdx];
```

```
const progress = ((currentIdx + 1) / questions.length) * 100;
const hasMap = q.map_lat !== null && q.map_lat !== undefined && q.map_lng !== n
const hasFlag = q.image_description && q.image_description.startsWith('FLAG_IMA
const flagIso = hasFlag ? q.image_description.replace('FLAG_IMAGE:', '') : null

return (
  <div className="quiz-container">
    <div className="quiz-header">
      <div style={{fontWeight:600}}>{chapter?.emoji} {chapter?.name}</div>
      <div className="quiz-progress"><div className="quiz-progress-bar" style={
      <div className="quiz-timer">{timeLeft}s</div>
      <div className="quiz-score">{score} pct</div>
    </div>
    <div className="question-card">
      <div className="question-meta">
        <span className="badge badge-topic">{q.topic}</span>
        <span className={`badge badge-${q.level.toLowerCase()}`}>{q.level}</spa
        <span style={{marginLeft:'auto', color:'var(--text-light)', fontSize:'0
      </div>
      <div className="question-text">{q.question_text}</div>
      {hasFlag ? (
        <FlagImage iso={flagIso} />
      ) : hasMap ? (
        <HintMap key={`map-${currentIdx}`} lat={parseFloat(q.map_lat)} lng={par
      ) : q.image_description && !q.image_description.startsWith('FLAG_IMAGE:')
        <div className="question-image">Indiciu vizual: {q.image_description}</
      ) : null}
      {q.type === 'multiple_choice' ? (
        <div className="options">
          {['option_a', 'option_b', 'option_c', 'option_d'].map((key, i) => {
            const opt = q[key];
            if (!opt) return null;
            const letter = ['A', 'B', 'C', 'D'][i];
            let cls = 'option-btn';
            if (showResult) {
              if (opt === q.correct_answer) cls += ' correct';
              else if (opt === selectedOption) cls += ' incorrect';
            }
            return (
              <button key={key} className={cls} disabled={showResult} onClick={
                <strong>{letter})</strong> {opt}
              </button>
            );
          })}
        </div>
      ) : (
        <div>
```

```
            <input type="text" className="fill-input" placeholder="Scrie raspunsu
            {!showResult && (
              <button onClick={() => handleAnswer(fillAnswer)} className="btn btn
            )}
            {showResult && (
              <div style={{marginTop:'1rem', padding:'1rem', borderRadius:'8px',
                {isCorrect ? 'Corect!' : `Raspunsul corect: ${q.correct_answer}`}
              </div>
            )}
          </div>
        )}
        {showResult && (
          <button onClick={nextQuestion} className="btn btn-primary next-btn">
            {currentIdx + 1 >= questions.length ? 'Vezi rezultatul' : 'Urmatoarea
          </button>
        )}
      </div>
    </div>
  );
}
```